

UNIVERSITY OF WAH(UOW)
DEPARTMENT OF COMPUTER
SCIENCE



Lab Task # 07

COURSE: Data Structure & Algorithm Lab_201L

SUBMITTED TO: Ms.Muniba Khan

SUBMITTED BY: Muhammad Talha_CS.3ndB

Registration NO: UW-24-CS-BS-101

Task # 01

```
struct Node
{
    int data;
    Node *prev;
    Node *next;
};

Node *head = NULL;
Node *tail = NULL;

void insertion_at_beg(int value)
{
    Node *newNode = new Node();
    newNode->data = value;

    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
    {
        head->prev = newNode;
    }
    head = newNode;
    cout << "Insertion at begging: " << value << endl;
}
```

```
void insertion_at_End(int value)
{
    Node *newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        newNode->prev = NULL;
        head = newNode;
        cout << "Insertion at End (First node): " << value << endl;
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;

    cout << "Insertion at End: " << value << endl;
}
```

```

void insertion_at_pos(int value, int pos)
{
    if (pos == 1)
    {
        insertion_at_beg(value);
        return;
    }

    Node *newNode = new Node();
    newNode->data = value;
    Node *temp = head;
    int c = 1;
    while (temp != NULL && c < pos - 1)
    {
        temp = temp->next;
        c++;
    }
    if (temp == NULL)
    {
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL)
    {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
    cout << "Insertion " << value << " at position" << pos << endl;
}

```

```

void display()
{
    if (head == NULL)
    {
        return;
    }
    Node *tail = head;
    while (tail->next != NULL)
    {
        tail = tail->next;
    }
    while (tail != NULL)
    {
        cout << tail->data << " <-> ";
        tail = tail->prev;
    }
    cout << "NULL";
}

```

```

int main()
{
    insertion_at_End(13);
    insertion_at_End(13);
    insertion_at_End(20);
    insertion_at_End(24);
    insertion_at_End(27);
    insertion_at_pos(45, 2);
    insertion_at_beg(5);
    insertion_at_End(37);
    display();
    return 0;
}

```

Output

```

Insertion at End (First node): 13
Insertion at End: 13
Insertion at End: 20
Insertion at End: 24
Insertion at End: 27
Insertion 45 at position2
Insertion at begging: 5
Insertion at End: 37
37 <-> 27 <-> 24 <-> 20 <-> 13 <-> 45 <-> 13 <-> 5 <-> NULL

```

Task # 02

```

struct Node {
    int data;
    Node* prev;
    Node* next;

    Node(int val) {
        data = val;
        prev = next = nullptr;
    }
};

class DoublyLinkedList {
private:
    Node* head;
    Node* tail;

public:
    DoublyLinkedList() {
        head = tail = nullptr;
    }
}

```

```

class DoublyLinkedList {
private:
    Node* head;
    Node* tail;

public:
    DoublyLinkedList() {
        head = tail = nullptr;
    }
    void insertAtBeginning(int val) {
        Node* newNode = new Node(val);

        if (head == nullptr) {
            head = tail = newNode;
        } else {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
        }
    }
}

```

```

    void insertAtEnd(int val) {
        Node* newNode = new Node(val);

        if (tail == nullptr) {
            head = tail = newNode;
            return;
        }

        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode; // update tail
    }
}

```

```

    void insertAfterPosition(int pos, int val) {
        if (head == nullptr) {
            cout << "List is empty.\n";
            return;
        }

        Node* temp = head;
        int count = 1;

        while (temp != nullptr && count < pos) {
            temp = temp->next;
            count++;
        }
    }
}

```

```

        if (temp == nullptr) {
            cout << "Position out of range.\n";
            return;
        }

        Node* newNode = new Node(val);
        newNode->next = temp->next;
        newNode->prev = temp;

        if (temp->next != nullptr) {
            temp->next->prev = newNode;
        } else {
            tail = newNode;
        }

        temp->next = newNode;
    }
}

```

```

int search(int key) {
    Node* temp = head;
    int pos = 1;

    while (temp != nullptr) {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    }

    return -1;
}

int countNodes() {
    int count = 0;
    Node* temp = head;

    while (temp != nullptr) {
        count++;
        temp = temp->next;
    }

    return count;
}

```

```

void deleteValue(int key) {
    Node* temp = head;

    while (temp != nullptr && temp->data != key) {
        temp = temp->next;
    }

    if (temp == nullptr) {
        cout << "Value not found.\n";
        return;
    }

    if (temp == head) {
        head = head->next;
        if (head != nullptr) head->prev = nullptr;
    }
    else if (temp == tail) {
        tail = tail->prev;
        tail->next = nullptr;
    }
    else {
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }

    delete temp;
    cout << "Value deleted.\n";
}

```

```

void forwardDisplay() {
    Node* temp = head;
    cout << "Forward: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << "\n";
}

void backwardDisplay() {
    Node* temp = tail;
    cout << "Backward: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << "\n";
}

};

```

```

int main() {
    DoublyLinkedList dll;

    dll.insertAtEnd(10);
    dll.insertAtEnd(20);
    dll.insertAtEnd(30);
    dll.insertAtEnd(40);
    dll.forwardDisplay();
    dll.backwardDisplay();
    cout << "\nInsert 25 after position 3:\n";
    dll.insertAfterPosition(3, 25);
    dll.forwardDisplay();
    cout << "\nSearching for value 25...\n";
    int pos = dll.search(25);
    if (pos != -1)
        cout << "Found at position: " << pos << endl;
    else
        cout << "Not found.\n";
    cout << "\nTotal nodes: " << dll.countNodes() << endl;
    cout << "\nInsert 50 at end using tail pointer:\n";
    dll.insertAtEnd(50);
    dll.forwardDisplay();
    return 0;
}

```

Output

```

Forward: 10 20 30 40
Backward: 40 30 20 10

Insert 25 after position 3:
Forward: 10 20 30 25 40

Searching for value 25...
Found at position: 4

Total nodes: 5

Insert 50 at end using tail pointer:
Forward: 10 20 30 25 40 50

```